Latihan 4.3

[singlylist.h]

```
#include <iostream>
#ifndef SINGLYLIST_H_INCLUDED
#define SINGLYLIST_H_INCLUDED

#define Nil NULL

using namespace std;

typedef int infotype;
typedef struct elmList *address;

struct elmList {
   infotype info;
   address next;
};

struct List {
   address first;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address p);
void printInfo(List L);
void insertFirst(List &L, address p);

#endif
```

[singlylist.cpp]

```
#include <iostream>
#include "singlylist.h"

using namespace std;

void createList(List &L) {
   L.first = Nil;
}

address alokasi(infotype x) {
   address p = new elmList;
   p->info = x;
   p->next = Nil;
   return p;
}
```

```
void dealokasi(address p) {
   delete p;
}

void insertFirst(List &L, address p) {
   if (L.first == Nil) {
      L.first = p;
   } else {
      p->next = L.first;
      L.first = p;
   }
}

void printInfo(List L) {
   address p = L.first;

   while (p != Nil) {
      cout << p->info << " ";
      p = p->next;
   }

   cout << endl;
}
```

[main.cpp]

```
#include <iostream>
#include "Singlylist.h"

using namespace std;

int main() {
   List L;
   address P1, P2, P3, P4, P5;

   createList(L);

   P1 = alokasi(2);
   insertFirst(L, P1);

   P2 = alokasi(0);
   insertFirst(L, P2);

   P3 = alokasi(8);
   insertFirst(L, P3);
```

```
    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    return 0;
}
```

